
MongoDB Capstone Project 2

Hospital Appointment & Billing Analytics

Database

```
use("hospital_db")
```

Collections:

- patients
 - doctors
 - appointments
-

1.Create Patients Collection

```
db.patients.insertMany([
  { patient_id: 1, name: "Aarav", city: "Hyderabad", age: 29, gender: "Male" },
  { patient_id: 2, name: "Priya", city: "Bangalore", age: 34, gender: "Female" },
  { patient_id: 3, name: "Rahul", city: "Mumbai", age: 41, gender: "Male" },
  { patient_id: 4, name: "Sneha", city: "Delhi", age: 26, gender: "Female" },
  { patient_id: 5, name: "Kiran", city: "Hyderabad", age: 37, gender: "Male" },
  { patient_id: 6, name: "Meera", city: "Chennai", age: 31, gender: "Female" }
])
```

2. Create Doctors Collection

```
db.doctors.insertMany([
  { doctor_id: 101, name: "Dr. Sharma", specialization: "Cardiology",
    consultation_fee: 1200, city: "Hyderabad" },
  { doctor_id: 102, name: "Dr. Iyer", specialization: "Dermatology",
    consultation_fee: 800, city: "Bangalore" },
  { doctor_id: 103, name: "Dr. Khan", specialization: "Orthopedics",
    consultation_fee: 1500, city: "Mumbai" },
  { doctor_id: 104, name: "Dr. Reddy", specialization: "Pediatrics",
    consultation_fee: 900, city: "Delhi" },
  { doctor_id: 105, name: "Dr. Mehta", specialization: "Neurology",
    consultation_fee: 2000, city: "Hyderabad" }
])
```

3. Create Appointments Collection

```
db.appointments.insertMany([
  { appointment_id: 1001, patient_id: 1, doctor_id: 101, visit_date: "2026-03-01",
    status: "Completed", bill_amount: 1200 },
  { appointment_id: 1002, patient_id: 2, doctor_id: 102, visit_date: "2026-03-02",
    status: "Completed", bill_amount: 800 },
  { appointment_id: 1003, patient_id: 1, doctor_id: 105, visit_date: "2026-03-04",
    status: "Pending", bill_amount: 2000 },
  { appointment_id: 1004, patient_id: 3, doctor_id: 103, visit_date: "2026-03-05",
    status: "Completed", bill_amount: 1500 },
  { appointment_id: 1005, patient_id: 5, doctor_id: 101, visit_date: "2026-03-07",
    status: "Cancelled", bill_amount: 1200 },
  { appointment_id: 1006, patient_id: 6, doctor_id: 104, visit_date: "2026-03-08",
    status: "Completed", bill_amount: 900 },
  { appointment_id: 1007, patient_id: 4, doctor_id: 104, visit_date: "2026-03-09",
    status: "Pending", bill_amount: 900 },
  { appointment_id: 1008, patient_id: 3, doctor_id: 105, visit_date: "2026-03-10",
    status: "Completed", bill_amount: 2000_id: 1008, patient_id: 3,
    doctor_id: 105, visit_date: "2026"]])
```

Assignment Tasks

Basic Queries

- . Display all documents from the **patients** collection.
`db.patients.find();`
 - . Display all documents from the **doctors** collection.
`db.doctors.find();`
 - . Display all documents from the **appointments** collection.
`db.appointments.find();`
 - . Display patients from **Hyderabad**.
`db.patients.find({city: "Hyderabad"});`
 - . Display doctors whose specialization is **Cardiology**.
`db.doctors.find({specialization: "Cardiology"});`
 - . Display appointments whose status is **Completed**.
`db.appointments.find({status: "Completed"});`
-

Filtering Queries

- . Display patients whose age is greater than **30**.
`db.patients.find({ age: { gt: 30 } });`
- . Display doctors whose consultation fee is greater than **1000**.
`db.doctors.find({ consultation_fee: { gt: 1000 } });`
- . Display doctors whose consultation fee is between **900 and 1600**.
`db.doctors.find({ consultation_fee: { $gte: 900, $lte: 1600 } });`
- . Display appointments whose bill amount is greater than **1000**.
`db.appointments.find({ bill_amount: { $gt: 1000 } });`

- . Display appointments whose status is not **Cancelled**.
`db.appointments.find({ status: { $ne: "Cancelled" } });`
 - . Display patients from **Hyderabad or Mumbai**.
`db.patients.find({ city: { $in: ["Hyderabad", "Mumbai"] } });`
 - . Display doctors located in **Hyderabad or Delhi**.
`db.doctors.find({ location: { $in: ["Hyderabad", "Delhi"] } });`
-

Projection Queries

- . Display only patient **name and city**.
`db.patients.find({}, { name: 1, city: 1, _id: 0 });`
 - . Display only doctor **name, specialization, and consultation_fee**.
`db.doctors.find({}, { name: 1, specialization: 1, consultation_fee: 1, _id: 0 });`
 - . Display only **appointment_id, status, and bill_amount**.
`db.appointments.find({}, { appointment_id: 1, status: 1, bill_amount: 1, _id: 0 });`
-

Sorting, Limit, Skip

- . Display doctors sorted by **consultation_fee ascending**.
`db.doctors.find().sort({ consultation_fee: 1 });`
- . Display doctors sorted by **consultation_fee descending**.
`db.doctors.find().sort({ consultation_fee: -1 });`
- . Display the **top 3 highest fee doctors**.
`db.doctors.find().sort({ consultation_fee: -1 }).limit(3);`
- . Display the **2 lowest fee doctors**.
`db.doctors.find().sort({ consultation_fee: 1 }).limit(2);`
- . Skip the first **2 patients** and display the rest.
`db.patients.find().skip(2);`

- . Display patients sorted by **age descending**.

```
db.patients.find().sort({ age: -1 });
```

Update Operations

- . Update the consultation fee of **Dr. Sharma** to **1300**.

```
db.doctors.updateOne({ name: "Dr. Sharma" }, { $set: { consultation_fee: 1300 } });
```

- . Add a new field `priority: "High"` to appointments with status **Pending**.

```
db.appointments.updateMany({ status: "Pending" }, { $set: { priority: "High" } });
```

- . Add a new field `available: true` to doctors located in **Hyderabad**.

```
db.doctors.updateMany({ location: "Hyderabad" }, { $set: { available: true } });
```

- . Change the city of patient **Meera** from **Chennai** to **Bangalore**.

```
db.patients.updateOne({ name: "Meera" }, { $set: { city: "Bangalore" } });
```

Delete Operations

- . Delete the doctor whose name is **Dr. Iyer**.

```
db.doctors.deleteOne({ name: "Dr. Iyer" });
```

- . Delete all appointments whose status is **Cancelled**.

```
db.appointments.deleteMany({ status: "Cancelled" });
```

- . Delete all patients whose city is **Delhi**.

```
db.patients.deleteMany({ city: "Delhi" });
```

Count Queries

- . Count total patients.

```
db.patients.countDocuments({});
```

- . Count doctors located in **Hyderabad**.

```
db.doctors.countDocuments({ location: "Hyderabad" });
```

. Count total **completed appointments**.

```
db.appointments.countDocuments({ status: "Completed" });
```

. Count patients from **Hyderabad**.

```
db.patients.countDocuments({ city: "Hyderabad" });
```

Aggregation Queries

. Find the **average consultation fee per specialization**.

```
db.doctors.aggregate([{$group: {_id: "$specialization", average_Fee: { $avg: "$consultation_fee" } } }]);
```

. Find the **maximum consultation fee**.

```
db.doctors.aggregate([{$group: {_id: null, max_Fee: { $max: "$consultation_fee" } } }]);
```

. Find the **minimum consultation fee**.

```
db.doctors.aggregate([{$group: {_id: null, min_Fee: { $min: "$consultation_fee" } } }]);
```

. Find the **total bill amount grouped by appointment status**.

```
db.appointments.aggregate([{$group: {_id: "$status", totalBill: { $sum: "$bill_amount" } } }]);
```

. Find the **total appointments handled by each doctor**.

```
db.appointments.aggregate([{$group: {_id: "$doctor_id", count: { $sum: 1 } } }]);
```

. Find the **total appointments per patient**.

```
db.appointments.aggregate([{$group: {_id: "$patient_id", count: { $sum: 1 } } }]);
```

. Find the **average patient age per city**.

```
db.patients.aggregate([{$group: {_id: "$city", avgAge: { $avg: "$age" } } }]);
```

. Find the **total bill amount generated per doctor**.

```
db.appointments.aggregate([
  {
    $lookup: {
      from: "doctors",
      localField: "doctor_id",
      foreignField: "doctor_id",
      as: "doctor_details"
    }
  },
  { $unwind: "$doctor_details" },
```

```
{
  $group: {
    _id: "$doctor_details.name",
    total_revenue: { $sum: "$bill_amount" }
  }
},
]);
```

. Find the **total bill amount generated per patient city**.

```
db.appointments.aggregate([
  {
    $lookup: {
      from: "patients",
      localField: "patient_id",
      foreignField: "patient_id",
      as: "patient_details"
    }
  },
  { $unwind: "$patient_details" },
  {
    $group: {
      _id: "$patient_details.city",
      total_billed: { $sum: "$bill_amount" }
    }
  },
  { $sort: { total_billed: -1 } }
]);
```

Multi-Collection Queries using \$lookup

. Display appointments with **patient details**.

```
db.appointments.aggregate([
  {
    $lookup: {
      from: "patients",
      localField: "patient_id",
      foreignField: "patient_id",
      as: "patient_details"
    }
  }
]);
```

```
    }  
  }  
]);
```

. Display appointments with **doctor details**.

```
db.appointments.aggregate([  
  {  
    $lookup: {  
      from: "doctors",  
      localField: " doctor_id",  
      foreignField: " doctor_id",  
      as: " doctor_details"  
    }  
  }  
]);
```

. Display **patient name and doctor name** for each appointment.

```
db.appointments.aggregate([  
  {  
    $lookup: { from: "patients", localField: "patient_id", foreignField: "patient_id", as: "p" }  
  },  
  {  
    $lookup: { from: "doctors", localField: "doctor_id", foreignField: "doctor_id", as: "d" }  
  },  
  { $unwind: "$p" },  
  { $unwind: "$d" },  
  {  
    $project: {  
      _id: 0,  
      appointment_id: 1,  
      patient_name: "$p.name",  
      doctor_name: "$d.name"  
    }  
  }  
]);
```

. Display **patient name, city, doctor name, specialization, appointment status, and bill amount**.

```
db.appointments.aggregate([  
  {  
    $lookup: { from: "patients", localField: "patient_id", foreignField: "patient_id", as: "p" }  
  },  
  {  
    $lookup: { from: "doctors", localField: "doctor_id", foreignField: "doctor_id", as: "d" }  
  },  
  { $unwind: "$p" },  
  { $unwind: "$d" },
```



```
{
  $project: {
    _id: 0,
    patient_name: "$p.name",
    city: "$p.city",
    doctor_name: "$d.name",
    specialization: "$d.specialization",
    status: 1,
    bill_amount: 1
  }
}
});
```

. Display all patients with their appointments.

```
db.patients.aggregate([
{
  $lookup: {
    from: "appointments",
    localField: "patient_id",
    foreignField: "patient_id",
    as: "patient_appointments"
  }
}
]);
```

. Display all doctors with the appointments assigned to them.

```
db.doctors.aggregate([
{
  $lookup: {
    from: "appointments",
    localField: "doctor_id",
    foreignField: "doctor_id",
    as: "assigned_appointments"
  }
}
]);
```

Advanced Aggregation

. Find **total revenue generated per doctor**.

```
db.appointments.aggregate([
  {
    $lookup: {
      from: "doctors",
      localField: "doctor_id",
      foreignField: "doctor_id",
      as: "doctor_info"
    }
  },
  { $unwind: "$doctor_info" },
  {
    $group: {
      _id: "$doctor_info.name",
      total_revenue: { $sum: "$bill_amount" }
    }
  }
]);
```

. Find **total revenue generated per specialization**.

```
db.appointments.aggregate([
  {
    $lookup: {
      from: "doctors",
      localField: "doctor_id",
      foreignField: "doctor_id",
      as: "doc"
    }
  },
  { $unwind: "$doc" },
  {
    $group: {
      _id: "$doc.specialization",
      total_revenue: { $sum: "$bill_amount" }
    }
  }
]);
```

. Find the **patient who spent the highest bill amount**.

```
db.appointments.aggregate([
{
  $group: {
    _id: "$patient_id",
    total_spent: { $sum: "$bill_amount" }
  }
},
{ $sort: { total_spent: -1 } },
{ $limit: 1 },
{
  $lookup: {
    from: "patients",
    localField: "_id",
    foreignField: "patient_id",
    as: "patient_details"
  }
},
{ $unwind: "$patient_details" },
{ $project: { name: "$patient_details.name", total_spent: 1, _id: 0 } }
]);
```

. Find the **doctor who handled the highest number of appointments**.

```
db.appointments.aggregate([
{ $group: { _id: "$doctor_id", count: { $sum: 1 } } },
{ $sort: { count: -1 } },
{ $limit: 1 },
{
  $lookup: {
    from: "doctors",
    localField: "_id",
    foreignField: "doctor_id",
    as: "doc"
  }
},
{ $unwind: "$doc" },
{ $project: { doctor_name: "$doc.name", appointment_count: "$count" } }
]);
```

. Find **total revenue generated from completed appointments only**.

```
db.appointments.aggregate([
  { $match: { status: "Completed" } },
  { $group: { _id: null, total_revenue: { $sum: "$bill_amount" } } }
]);
```

. Find the **city with the highest number of appointments**.

```
db.appointments.aggregate([
  {
    $lookup: {
      from: "patients",
      localField: "patient_id",
      foreignField: "patient_id",
      as: "patient"
    }
  },
  { $unwind: "$patient" },
  { $group: { _id: "$patient.city", appointment_count: { $sum: 1 } } },
  { $sort: { appointment_count: -1 } },
  { $limit: 1 }
]);
```

. Find the **city with the highest bill amount generated**.

```
db.appointments.aggregate([
  {
    $lookup: {
      from: "patients",
      localField: "patient_id",
      foreignField: "patient_id",
      as: "patient"
    }
  },
  { $unwind: "$patient" },
  { $group: { _id: "$patient.city", total_revenue: { $sum: "$bill_amount" } } },
  { $sort: { total_revenue: -1 } },
  { $limit: 1 }
]);
```

. Find the **specialization with the highest average bill amount**.

```
db.appointments.aggregate([
  {
    $lookup: {
      from: "doctors",
      localField: "doctor_id",
      foreignField: "doctor_id",
      as: "doc"
    }
  },
  { $unwind: "$doc" },
  {
    $group: {
      _id: "$doc.specialization",
      avg_bill: { $avg: "$bill_amount" }
    }
  },
  { $sort: { avg_bill: -1 } },
  { $limit: 1 }
]);
```
